Plan de tests — Triangulator

Pour organiser les tests, on découpe le `Triangulator` en quelques classes/modules, chacun avec une responsabilité précise. Les noms ci-dessous sont indicatifs (ils pourront changer une fois qu'on commence à coder), mais le découpage des responsabilités doit rester le même :

- `PointSet` : représente un ensemble de points, sait se décoder depuis le format binaire et s'encoder vers ce format.
- `Triangles` : représente le résultat d'une triangulation (sommets + indices de triangles), sait s'encoder vers le format binaire.
- Algorithme de triangulation : prend un `PointSet` en entrée, renvoie les triangles calculés.
- Client `PointSetManager` : sait appeler le service `PointSetManager` en HTTP pour récupérer un `PointSet` à partir d'un `pointSetId`.
- Route API (`/triangulation/{pointSetId}`) : fait le lien entre les trois éléments précédents et construit la réponse HTTP.

Chaque classe a ses propres tests unitaires (elle est testée seule, sans dépendre des autres), présentés sous forme de tableau. La route API est testée à part, en bout en bout, avec des exemples de messages JSON plutôt qu'un tableau (trop de combinaisons pour que ça tienne dans un tableau lisible).

Vocabulaire des exceptions/codes utilisé dans tout le document :

Cas	Exception Python levée	Code HTTP	`code` JSON
Buffer `PointSet` invalide/tronqué	`PointSetDecodingError`	400 (requête entrante) / 500 (réponse du `PointSetManager`)	`INVALID_POINT_SET_FORMAT`
Indice de triangle hors bornes	`InvalidTriangleIndexError`	500	`INVALID_TRIANGLE_INDEX`
Triangulation impossible	`TriangulationError`	500	`TRIANGULATION_FAILED`
`pointSetId` mal formé	`InvalidPointSetIdError`	400	`INVALID_POINT_SET_ID`
`pointSetId` inconnu	`PointSetNotFoundError`	404	`NOT_FOUND`
`PointSetManager` injoignable/timeout	`PointSetManagerUnavailableError`	503	`POINT_SET_MANAGER_UNAVAILABLE`

1. Classe `PointSet`

Sujet	Problème testé	Résultat attendu
Round-trip	Encoder un `PointSet` valide puis le décoder	On retrouve exactement les mêmes points, dans le même ordre
Ensemble vide	`PointSet` avec 0 point	Décodage réussi, liste de points vide (pas une erreur)
Buffer trop court	Moins de 4 bytes, ou coupé au milieu d'un point	Lève `PointSetDecodingError`
Taille incohérente	Le nombre de points annoncé ne correspond pas au nombre de bytes restants	Lève `PointSetDecodingError`

2. Classe `Triangles`

Sujet	Problème testé	Résultat attendu
Round-trip	Encoder un ensemble de triangles puis le décoder	On retrouve les mêmes sommets et les mêmes triangles
Aucun triangle	Sommets présents, 0 triangle	Encodage/décodage valide, 0 triangle en sortie
Indice hors bornes	Un triangle référence un indice de sommet qui n'existe pas	Lève `InvalidTriangleIndexError`

3. Algorithme de triangulation

Sujet	Problème testé	Résultat attendu
Cas simple	3 points non alignés	Exactement 1 triangle, contenant les 3 points
Carré	4 points formant un carré	Exactement 2 triangles couvrant le carré
Pas assez de points	0, 1 ou 2 points	Lève `TriangulationError` (pas de résultat inventé)
Points alignés	3+ points colinéaires	Lève `TriangulationError` (aucun triangle valide possible)
Points dupliqués	Deux points avec les mêmes coordonnées	Comportement géré sans planter (ignoré ou erreur, à trancher)
Indices valides	N'importe quel résultat non trivial	Chaque indice utilisé dans un triangle existe bien dans le `PointSet` d'entrée

4. Client `PointSetManager`

Testé avec un `PointSetManager` simulé, pas le vrai service.

Sujet	Problème testé	Résultat attendu
Appel correct	Requête vers le `PointSetManager` simulé	`GET /pointset/{id}` appelé avec exactement l'ID demandé
Id inconnu	Le simulé répond 404	Lève `PointSetNotFoundError`
Service indisponible	Le simulé ne répond pas / timeout	Lève `PointSetManagerUnavailableError` (pas de blocage indéfini)
Réponse corrompue	Le simulé renvoie un `PointSet` binaire invalide	Le `PointSetDecodingError` de la classe `PointSet` remonte sans être avalé
Réponse 400 inattendue	Le simulé répond 400 (cas documenté par `point_set_manager.yml`)	Géré sans planter, traité comme `PointSetManagerUnavailableError`

5. Route API `/triangulation/{pointSetId}`

Tests bout en bout, avec un `PointSetManager` simulé. Plutôt qu'un tableau, voici les messages JSON attendus pour chaque cas (le format vient du schéma `Error` de `triangulator.yml` : `code` + `message`).

Succès — `pointSetId` valide et existant :

- Code `200`, `Content-Type: application/octet-stream`.
- Corps : `Triangles` binaire, décodable, cohérent avec le `PointSet` simulé.

`pointSetId` mal formé (pas un UUID) — code `400` :

```
{
  "code": "INVALID_POINT_SET_ID",
```

```
  "message": "'abc' is not a valid pointSetId (expected a UUID)."
}
```

`pointSetId` **inconnu** (le `PointSetManager` répond 404) — code `404` :

```
{
  "code": "NOT_FOUND",
  "message": "No PointSet found for id '123e4567-e89b-12d3-a456-426614174000'."
}
```

`PointSetManager` **indisponible** — code `503` :

```
{
  "code": "POINT_SET_MANAGER_UNAVAILABLE",
  "message": "Could not reach the PointSetManager service."
}
```

Triangulation impossible (points insuffisants ou alignés) — code `500` :

```
{
  "code": "TRIANGULATION_FAILED",
  "message": "Triangulation could not be computed for the given point set."
}
```

`PointSet` **reçu mais binaire corrompu** — code `500` :

```
{
  "code": "INVALID_POINT_SET_FORMAT",
  "message": "The PointSet returned by PointSetManager is not valid."
}
```

6. Tests de performance (à part)

Le sujet demande de mesurer les perfs de la triangulation et de la conversion binaire, séparément des autres tests.

- Mesurer le temps pris par l'algorithme de triangulation avec beaucoup de points (par exemple 1000, 10 000 points).
- Mesurer le temps pris par le décodage/encodage de `PointSet` et `Triangles` avec de gros volumes de données.
- Pas un pass/fail strict : plutôt relever le temps mesuré, et repérer si le temps explose anormalement quand le nombre de points augmente.
- Ces tests seront marqués à part (marker `pytest`) pour pouvoir les exclure avec `make unit_test` et les lancer seuls avec `make perf_test`.